Resumen: Semana 8 IIC2233

1 Funciones Generadoras en Python

Las funciones generadoras permiten iterar sobre datos sin almacenarlos completamente en memoria, usando la sentencia `yield`. A diferencia de `return`, `yield` pausa la ejecución y la retoma desde el mismo punto en la siguiente llamada.

1.1 Ejemplos básicos

1.1.1 Generador decreciente

```python
def conteo_decreciente(n):
    while n > 0:
        yield n
        n -= 1
```

1.1.2 Fibonacci infinito

```python
def fibonacci():
    a, b = 0, 1
    while True:
        yield b
        a, b = b, a + b
```

1.1.3 Lectura de archivos línea por línea

```python
def cargar_archivo(path):
    with open(path, encoding='utf-8') as file:
        for line in file:
            yield line
```

1.2 Generadores con listas

```python
def maximo_acumulativo(valores):
    max_ = float('-inf')
    for valor in valores:
        max_ = max(valor, max_)
        yield max_
```

1.3 Hacer iterable una clase propia

```python
class IterableListaNumeros:
    def __init__(self, objeto):
        self.objeto = objeto.copy()
    def __iter__(self):
        while self.objeto:
            yield self.objeto.pop(0)
```

1.4 Comunicación con `send()`

```python
def funcion_generadora_send():
    contador = 0
    while True:
        valor_recibido = yield contador
        if valor_recibido is None:
            valor_recibido = 0
        contador += valor_recibido
```

1.5 `yield from`: Composición de generadores

```python
def generador_padre():
    yield from range(5)
```

```python
def generador_hijo():
    for i in range(5):
        yield i
def generador_padre():
    yield from generador_hijo()
```

1.6 ¿Qué hace que una función sea generadora?

Toda función que contenga `yield` es considerada generadora, aunque nunca se ejecute.

```python
def f():
    if False:
        yield 1
    return 1
```

Conclusión: Las funciones generadoras son herramientas poderosas para crear iteradores personalizados, eficientes y flexibles.

2 Funciones que Retornan Generadores en Python

Son funciones que devuelven objetos generadores, estructuras que producen valores uno a uno bajo demanda, sin almacenar todos los resultados en memoria.

Se dividen en dos grupos:

- Funciones que necesitan otra función como argumento.
- Funciones que no necesitan otra función.

2.1 Funciones que necesitan otra función

2.2 map(función, iterable)

Aplica una función a cada elemento de uno o más iterables.

```python
map(lambda x: x**2, [1, 2, 3])  #      [1, 4, 9]
```

2.3 filter(función_booleana, iterable)

Filtra elementos que cumplen una condición.

```python
filter(lambda x: x % 2 == 0, [1, 2, 3, 4])  #      [2, 4]
```

2.4 reduce(función, iterable[, inicializador])

Aplica una función acumulativa sobre los elementos de un iterable.

```
from functools import reduce
reduce(lambda x, y: x + y, [1, 2, 3, 4])  #     10
```

2.5 Funciones lambda

Permiten definir funciones simples en una sola línea.

```
lambda x: x + 1
```

2.6 Funciones que no necesitan otra función

2.7 enumerate(iterable)

Devuelve pares (índice, elemento).

```
for i, val in enumerate(['a', 'b', 'c']):
    print(i, val)
```

2.8 zip(iterable1, iterable2, ...)

Agrupa elementos por posición en tuplas.

```
zip([1, 2], ['a', 'b'])  #     [(1, 'a'), (2, 'b')]
```

También puede invertirse con zip(*zipped).

```
zipped = zip([1, 2], ['a', 'b'])
list(zip(*zipped))  #     [(1, 2), ('a', 'b')]
```

2.9 Precauciones

- **reduce** con un solo elemento no aplica la función.
- Operaciones sensibles al orden pueden dar resultados inesperados.
- Algunos iterables como `set` no garantizan orden.

3 Aplicaciones de funciones generadoras en Python

Este apartado resume el contenido del documento sobre aplicaciones prácticas de funciones generadoras en Python, incluyendo funciones `built-in`, librerías estándar y casos aplicados.

3.1 Funciones `built-in` en Python

Python incluye funciones integradas que simplifican operaciones sobre objetos iterables. Algunas destacadas son:

- `len()`: Retorna el número de elementos de un contenedor.
- `__getitem__()`: Permite acceder a elementos mediante índices o llaves.
- `reversed()`: Invierte el orden de una secuencia.
- `sum()`, `min()`, `max()`, `all()`, `any()`: Operaciones matemáticas y booleanas sobre iterables.

3.1.1 Ejemplo de clase personalizada con `len`

```
class MiLista(list):
    def __len__(self):
        return len(set(self))
```

3.2 Librerías estándar

3.2.1 `itertools`

Proporciona herramientas para trabajar con iteradores:

- `count()`, `cycle()`, `repeat()`
- `product()`, `permutations()`, `combinations()`

3.2.2 `collections`

Ofrece estructuras como:

- `namedtuple`, `deque`, `defaultdict`, `Counter`

3.2.3 Ejemplo con `Counter`

```
from collections import Counter
texto = 'supercalifragilisticoespialidoso'
contador = Counter(texto)
print(contador.most_common(3))
```

3.3 Casos aplicados

3.3.1 Consultas anidadas

Uso de `filter` y `map` para filtrar lenguas vulnerables en India:

```
hablantes_totales = sum(
    map(lambda l: l.speakers,
        filter(lambda l: l.country == 'India' and l.endangerment == 'Vulnerable',
               generador_lenguas_extintas()))
)
```

3.3.2 Unión de generadores

Combinación de generadores con `product`:

```
from itertools import product
generador_1 = (randint(0, 20) for _ in range(10))
generador_2 = (randint(20, 50) for _ in range(10))
combinaciones = product(generador_1, generador_2)
pares = filter(lambda x: (x[0] + x[1]) % 5 == 0, combinaciones)
```

3.3.3 Agrupación con diccionarios

- Por comprensión de diccionarios.
- Con `Counter`.
- Usando `reduce`.

3.3.4 Comparación de eficiencia

Se comparan tiempos y uso de memoria entre:

- Lectura línea por línea vs. lectura completa.
- Uso de `namedtuple` vs. clases personalizadas.
- Unión de datos con generadores vs. manual.

Conclusión: El uso de generadores mejora la eficiencia y reduce el uso de memoria, especialmente en grandes volúmenes de datos.